Windows 本地 AI 伺服器

建置全攻略

硬體架構理解 × LM Studio 安裝與設定 × API 伺服器架設

自然語言優化 × 效能調校與除錯

適用環境：Windows 10 / 11 | LM Studio 0.3.x 以上 | 任何具備 GPU 的桌機或筆電

前言：這份指南能幫你做什麼？

這份指南涵蓋從零開始在 Windows 電腦上架設本地 AI 伺服器的完整流程，並說明每個設定背後的技術原理，讓你不只知道「怎麼做」，也明白「為什麼這樣做」。

本文中使用的所有工具，在 2026 年 3-4 月時均為開源狀態。只要硬體條件達標，軟體的取得基本完全免費。（若串接 Gemini, Claude 等 API 則例外）

章節	內容摘要
第一章：核心術語	參數、量化、上下文窗口、GGUF 格式的詳細解說
第二章：硬體與模型匹配	VRAM / RAM 的速度差異，以及各硬體等級的建議配置
第三章：伺服器架設流程	從安裝 LM Studio 到啟動 API Server 的完整步驟
第四章：API 整合與自然語言優化	OpenAI 協定、System Prompt 調教、Chat Template 選擇
第五章：效能優化設定	LM Studio 五個關鍵設定的邏輯與建議值
第六章：除錯手冊	常見錯誤與速度異常的完整排查流程
第七章：操作心法	核心哲學、快速檢查清單與長期使用建議

第一章：AI 核心術語解析

開始設定之前，先建立對 AI 模型的基本認識，讓後續的每一個設定決策都有理論依據。

1.1　模型規模：參數（Parameters）

AI 模型是由數十億個神經網路連接組成的，「參數」代表這些連接的數量，以 B（Billion，十億）為單位。參數越多，模型通常越聰明，但對記憶體的需求也越高。

參數規模	特性	適合場景
1B – 3B	迷你模型，邏輯較弱	手機、文書筆電、簡單指令
7B – 9B	主流標準，CP 值最高	日常對話、基礎代碼撰寫
14B – 20B	邏輯能力顯著提升	複雜代碼 Debug、資料分析
30B – 35B	具備強大推理能力	架構設計、長篇邏輯推演
70B+	邏輯接近頂尖商業模型	旗艦任務，需高階工作站

1.2 量化技術（Quantization）——讓大模型瘦身的魔法

量化是將原始高精度數值（16-bit）壓縮成低精度（如 4-bit）的技術，讓家用電腦也能執行大型模型。壓縮率越高，檔案越小、速度越快，但模型智商也略有損失。

格式	位元數	體積（相對 FP16）	品質損失	使用建議
FP16 / BF16	16-bit	100%（原始大小）	無	訓練用，本地推論通常無法負擔
Q8_0	8-bit	約 52–55%	幾乎無損	有足夠 VRAM 時使用，品質最接近原始
Q6_K	6-bit	約 37–38%	極微，幾乎不可感知	★ 強烈推薦的黃金平衡點
Q4_K_M	4-bit	約 26–28%	約 3–5%	VRAM 不足時的首選，速度最快
IQ3_M / IQ2_M	2–3-bit	約 19–26%	明顯，但優於同位元的 Q 系列	極端壓縮，適合超低 VRAM 場景

量化格式補充說明

Q6_K 的原理：將每個 16-bit 浮點數壓縮為 6-bit，並附加分組縮放因子（K 代表 K-quants 演算法）。實際體積約為原始的 37–38%，比單純的「縮小一半」更有效率。

Q4_K_M 中的 M 代表「Medium」——4-bit 中針對模型的重要層（如 attention 層）使用較高精度的混合方案，比純 Q4_0 品質更穩定，是目前本地部署最廣泛使用的格式。

IQ 系列（Importance Quantization）：根據每個參數對模型輸出的重要程度分配不同精度，在相同體積下比傳統 Q 系列保留更多模型能力，適合極端壓縮場景。

1.3 模型格式：GGUF vs. Safetensors

- **GGUF 格式**：由 llama.cpp 社群開發，專為「本地推論」設計。單一檔案封裝、支援 CPU/GPU 混合運算、載入速度快，是 LM Studio 的原生格式，也是 Windows 使用者的首選。
- **Safetensors 格式**：Hugging Face 的標準訓練格式，通常分割為十幾個分片檔案。主要用於模型訓練或雲端部署，不建議直接載入 LM Studio，需先轉換為 GGUF。

1.4 上下文窗口（Context Window / n_ctx）

上下文窗口是 AI 的「短期記憶」，包含你輸入的所有內容（文件、問題）以及模型的完整回答歷史。LM Studio 在載入模型時會依據 n_ctx 的設定值，預先在 VRAM 中分配一塊固定大小的 KV Cache 空間——這塊空間即使對話內容還很少，也會佔著不放。因此，n_ctx 設得越大，一開始就佔用越多顯存。

⚠ Context 設定的關鍵影響
Context 佔用的是「KV Cache」顯存，會與「模型主體」競爭同一塊 VRAM。
一次貼入多個代碼檔案，Context 消耗可能暴增，直接將模型擠出顯卡。
建議：一般對話設 4096（4k），代碼分析設 8192（8k）；無特殊需求不要超過這個範圍。

1.5 進階概念：MoE（Mixture of Experts，混合專家模型）

部分模型採用 MoE 架構，例如 Qwen3-30B-A3B。雖然總參數量為 30B，但模型內部被劃分為許多組「專家（Expert）」子網路。每次處理一個 token 時，路由機制（Router）會動態選擇其中最相關的少數幾組專家（約 3B 的參數量）來計算，其他專家在這個 token 上不參與計算。這讓它具備大模型的知識廣度，運算量卻接近小模型，是本地部署的高效率選項。但要注意：MoE 模型仍需將所有 30B 的參數載入記憶體，只是實際運算量少，因此記憶體需求並未等比縮減。

第二章：硬體資源的調度邏輯

在 Windows 上跑 AI，本質上是「顯存（VRAM）」與「系統記憶體（RAM）」的競賽。理解各層記憶體的速度差異，是一切優化設定的基礎。

2.1 三層記憶體架構

記憶體層級	頻寬（速度）	角色定位與特性
VRAM （顯卡記憶體）	約 912 GB/s （RTX 3080 Ti）	AI 的高速辦公桌。模型必須盡量住在這裡才能達到最高速度。
System RAM （主機記憶體）	DDR4 約 50 GB/s， DDR5 約 80–100 GB/s	VRAM 的後援區。速度比 VRAM 慢 10–20 倍，容量大但會造成明顯掉速。
Shared GPU Memory	取決於系統記憶體頻寬 （DDR4/5）	Windows 在 VRAM 滿溢時自動啟用。數據實際上存放在系統 RAM，GPU 透過 PCIe 通道存取，速度瓶頸在於 RAM 頻寬（50–100 GB/s），應竭力避免。

顯存溢出警告

當模型主體 + KV Cache 超過 VRAM 上限，數據溢出到 System RAM，速度會從 30 t/s 驟降到 1–5 t/s（慢約 10–20 倍）。

診斷方式：開啟 Windows 工作管理員 → 效能 → GPU，觀察「共用 GPU 記憶體（Shared）」欄位。只要出現數值，即代表溢出已發生。

目標：讓 Shared GPU Memory 保持為 0。

2.2 硬體等級與模型配置建議

下表以 Q4_K_M 量化為基準，協助你根據手邊的硬體選擇最合適的模型規模：

硬體等級	VRAM / RAM 規格	建議模型	預期速度
文書筆電（無顯卡）	無專用 VRAM / 8–16GB RAM	1B–3B Q4 或 Q6	3–10 t/s（CPU 模式）
Apple Silicon（MacBook）	Unified Memory 8–16GB（CPU/GPU 共用）	7B Q4（8GB）/ 13B Q4（16GB）	20–35 t/s（Metal 加速）
入門筆電（獨顯）	4–6GB VRAM / 16GB RAM	7B–8B Q4	15–25 t/s
中階電競筆電	6GB VRAM / 16GB RAM	7B Q4_K_M	20–35 t/s
高階電競筆電	12GB VRAM / 32GB RAM	9B Q6 或 35B Q3	30–50 t/s 或 3–8 t/s
頂級桌機	12GB VRAM / 96GB RAM	9B Q6 或 35B Q4	30–50 t/s 或 1–5 t/s
高階工作站	24GB+ VRAM	30B–35B Q4/Q5	15–25 t/s

硬體優勢：96GB RAM + 12GB VRAM

速度模式：9B (Q6) + Context 8k + KV Cache Q4_0，純 VRAM 運算，享受 30–50 t/s 的流暢體驗。

推理品質模式：35B (Q4) 善用 96GB RAM 的超大空間，以 1–5 t/s 的速度處理需要深度推理的複雜任務——這是大多數 12GB 顯卡使用者辦不到的。

第三章：本地 AI 伺服器架設流程

本章說明如何從零開始安裝 LM Studio、下載模型，並啟動 API 伺服器，讓其他應用程式（如 Open WebUI、自訂腳本）能夠透過網路呼叫本地 AI。

3.1 環境前置需求

開始前請確認
作業系統：Windows 10 (21H2 以上) 或 Windows 11
顯示卡驅動：至官方網站下載並安裝最新版 NVIDIA 驅動（若使用 NVIDIA GPU）
Python 3.12（64-bit）：LM Studio 的部分延伸工具（如 llama-cpp-python 或自訂腳本）依賴 Python。3.13 以上版本目前與部分套件不相容，請務必安裝 3.12（64-bit）並勾選「Add Python to PATH」
磁碟空間：建議預留至少 20GB 給模型檔案（7B Q4 模型約 4–5GB，35B Q4 約 20GB）
Node.js（選用）：若需要使用 OpenClaw 或其他 JavaScript 插件才需安裝

3.2 安裝 LM Studio

Step 1	下載安裝程式
	前往 LM Studio 官方網站 lmstudio.ai，點擊「Download for Windows」。
	下載完成後執行安裝程式（.exe），依照預設選項完成安裝即可。
	安裝完成後啟動 LM Studio，首次啟動會自動完成初始設定。

Step 2	確認 GPU 加速已啟用
	啟動後，點擊左下角齒輪圖示進入「Settings」。
	在「Hardware」頁面確認顯示卡已被正確識別（應顯示你的 GPU 名稱與 VRAM 大小）。
	若顯示「No GPU detected」，請先更新顯示卡驅動後重啟電腦再試。

3.3 下載模型

Step 3	搜尋並下載 GGUF 模型
	點擊左側選單的「Discover」（望遠鏡圖示），進入模型搜尋頁面。
	在搜尋列輸入模型名稱，例如：「Qwen3」或「Llama 3」。
	選擇適合你硬體的量化版本（參考第二章的表格）。建議優先選擇 Q6_K 或 Q4_K_M。
	點擊「Download」開始下載。模型會儲存在 LM Studio 的預設路徑（通常為 C:\Users\你的帳號\.lmstudio\models）。

在 Hugging Face 手動下載模型
若 LM Studio 內建搜尋找不到特定模型，可至 huggingface.co 搜尋。
篩選條件：在模型頁面點「Files and versions」，找副檔名為 .gguf 的檔案下載。
下載後，在 LM Studio 中點「My Models」→「Add Model Folder」，選擇放置 .gguf 檔案的資料夾即可匯入。

3.4 載入模型並初步測試

Step 4	載入模型
	點擊左側選單的「Chat」，在頂部下拉選單中選擇剛下載的模型。
	第一次載入需要等待幾秒至數十秒（視模型大小）。載入完成後，底部狀態列會顯示模型名稱與當前的 t/s 速度。
	在對話框輸入一句話測試，確認模型有正常回應。

Step 5	確認效能指標正常
	觀察對話框底部的統計數字：t/s（每秒輸出字元數）應在 10 以上才算正常，30 以上為良好。
	同時開啟 Windows 工作管理員（Ctrl+Shift+Esc）→「效能」→「GPU」，確認「共用 GPU 記憶體（Shared）」為 0。
	若 t/s 過低或 Shared 有數值，請先依照第五章調整設定再繼續。

3.5 啟動 API 伺服器

LM Studio 內建一個相容於 OpenAI API 格式的本地伺服器，讓你可以用任何支援 OpenAI 協定的工具（如 Open WebUI、自訂 Python 腳本、Cursor IDE）呼叫本地模型。

Step 6	開啟 Local Server 介面

	點擊左側選單的「Local Server」（伺服器圖示）。
	確認頂部的模型下拉選單已選好你要對外提供的模型。
	預設埠號為 1234，一般不需更改。若有埠號衝突，可在此修改。

Step 7	啟動伺服器
	點擊「Start Server」按鈕。按鈕變綠並顯示「Server Running」即表示啟動成功。
	伺服器的完整 API 位址為：http://localhost:1234/v1
	此時你的本地 AI 伺服器已對本機其他程式開放，可以開始接受請求。

Step 8	測試 API 連線
	開啟 PowerShell 或命令提示字元，貼上以下指令測試伺服器是否正常回應：
	curl http://localhost:1234/v1/models
	若看到 JSON 格式的模型清單，即代表伺服器運作正常。
	若需要從其他裝置（如同區網的手機或筆電）連線，將 localhost 替換為你電腦的區網 IP（例如 192.168.1.100）即可。

常見整合應用
Open WebUI：提供類 ChatGPT 的網頁介面，可設定連線至 http://localhost:1234/v1 使用本地模型。
Cursor / VS Code Copilot 替代插件：在 API Base URL 填入 http://localhost:1234/v1，API Key 可填任意字串（本地不驗證）。
自訂 Python 腳本：使用 openai 套件，設定 base_url='http://localhost:1234/v1'，api_key='lm-studio' 即可調用。

第四章：API 整合與「讓 AI 說人話」

伺服器架設完成後，下一步是讓外部工具能正確與它溝通，並確保 AI 以自然語言回應，而不是輸出一堆程式碼格式。本章涵蓋 OpenAI 相容協定、System Prompt 調教、對話模板選擇，以及常見的接入問題解法。

4.1 為什麼要使用 OpenAI 相容協定？

OpenAI 制定了一套被全球 AI 工具廣泛採用的「標準語言」（API 協定）。LM Studio 的本地伺服器完整相容這套協定，這帶來兩個核心好處：

- **無縫接軌現有工具：**所有原本為 ChatGPT 設計的程式（Python 框架、OpenClaw、Cursor IDE），不需更改任何代碼，只需把伺服器地址換成本地 URL 即可直接使用。
- **格式統一：**協定規定對話訊息必須包含兩個欄位：role（角色，分為 system、user、assistant）與 content（訊息內容），讓各工具之間的訊息格式完全一致。

關鍵連線資訊
API Base URL：http://localhost:1234/v1（這就是你的「本地 OpenAI 伺服器」地址）
API Key：本地伺服器不驗證金鑰，但多數框架會檢查格式。建議填入 lm-studio 或 sk-123456。
Model Name（重要陷阱）：框架傳送的模型名稱必須與 LM Studio 當前載入的名稱完全一致。正確名稱可在 LM Studio「Local Server」頁面的模型下拉選單中確認，例如 qwen2.5-9b-instruct-q6_k。名稱不符會觸發「Model not found」錯誤。

4.2 System Prompt——賦予 AI 說話的靈魂

當模型接上外部框架後，它往往會過度專注於「執行指令」，忘了用自然語言與你溝通。System Prompt 是解決這個問題的關鍵——它在每次對話開始前告訴模型「你是誰、你應該怎麼說話」。

設定位置：LM Studio「Chat」頁面右側的「System Prompt」欄位，或外部框架（如 OpenClaw）設定檔中的 system_prompt 欄位。

推薦的自然語言 System Prompt（可直接複製使用）
You are an AI assistant powered by a local language model.
1. Use natural language to communicate with the user at all times.
2. Even if you use a tool or function, explain what you are doing in plain text first.
3. Always respond in Traditional Chinese (Taiwan) unless the user explicitly asks otherwise.
4. Do not output raw JSON or tool-call syntax as your primary response.

4.3 對話模板（Chat Template）——溝通的語法規則

Chat Template 是一套「標籤語法」，告訴模型訊息的邊界：哪裡是問題的開始、哪裡是回答的結束。選錯模板，模型就像聽不懂句子的外國人，會產生嚴重的輸出混亂。

模板名稱	適用模型系列	說明
ChatML	Qwen 全系列、Mistral v0.3+、Mistral Nemo	使用 <|im_start|> / <|im_end|> 標籤。Qwen 模型的首選格式，也是目前最廣泛採用的格式。
Llama 3	Meta Llama 3 / 3.1 / 3.2 系列	使用 <|begin_of_text|>、<|eot_id|> 等標籤，與 Llama 2 格式不相容。
Mistral	Mistral v0.1 / v0.2、早期 Mixtral	使用 [INST] / [/INST] 標籤。注意：Mistral v0.3 以後已改用 ChatML。
Alpaca	早期指令微調模型（2023 年以前）	使用 ### Instruction / ### Response 格式。現代模型基本不再使用。

⚠ 選錯模板的後果

若為 Qwen 模型選擇了 Llama 或 Alpaca 模板，模型會無法正確辨識訊息邊界，出現：

- 胡言亂語或重複輸出同樣的句子
- 輸出無效的 JSON 或工具調用格式
- 對話角色錯亂（模型同時扮演 user 和 assistant）

設定路徑：LM Studio → Chat → 右側「Prompt Format」下拉選單 → 選擇對應的模板。

4.4 進階：解決「只出代碼、不說人話」

即使 System Prompt 和模板都設定正確，仍可能遇到模型輸出 {"tool_use": ...} 而非自然語言的情況。此時請調整以下兩個參數：

設定項目	建議值	原因說明
Temperature（溫度）	0.6–0.8	控制輸出的隨機性。0.7 適合日常對話，語言自然流暢；0.1–0.2 適合代碼生成，要求精確執行。設為 0 時採用 Greedy Decoding（每次選概率最高的詞），確定性最高，但過低會讓語言變得生硬，且在部分場景下仍可能因浮點運算差異產生微小變化。
Stop Strings（停止符號）	<|im_end|>	告訴模型何時停止輸出。設定為 <|im_end|>（ChatML 格式）或 \nuser，可防止模型在回答完畢後繼續「自我扮演」替你說話，或陷入無止境輸出廢話的迴圈。

4.5 驗證整合是否成功

完成以上設定後，可用以下方法快速確認一切正常：

✅ 成功 vs 失敗的判斷標準
✅ **成功狀態：**你問「你好」，AI 用繁體中文回覆「你好！我是你的本地助理，今天有什麼可以幫你的嗎？」
❌ **失敗狀態 1：**AI 回覆一串 JSON（{"tool_use": ...}）→ 檢查 System Prompt 與 Chat Template。
❌ **失敗狀態 2：**框架回報「Model not found」→ 確認框架中填寫的 model 名稱與 LM Studio 載入的名稱完全一致。
💡 **實戰心得：**即使在追求極致精確的代碼開發情境下，也要在 System Prompt 中保留「先用文字說明你的做法」的指示。這能讓你清楚掌握 AI 的思考過程，大幅降低除錯難度。

第五章：LM Studio 效能優化設定

載入模型後，正確設定以下五個參數，是讓 AI 跑得快且穩定的關鍵。每個設定都有明確的技術邏輯，而不是盲目調整數字。

設定項目	建議值	原因說明
GPU Offload	-1（Max）	模型由數十個「層（Layer）」組成，每一層都可以分配給 GPU 或 CPU 計算。設為 -1 表示強制所有層由 GPU 處理；若顯存確實不足，LM Studio 會自動將多餘的層降回 CPU。設定過低會讓部分運算卡在 CPU，速度降至 5–8 t/s。
Context Length	8192（8k）	AI 的短期記憶區大小。開 32k 可能預扣高達 4GB 顯存，將模型主體擠出顯卡；開 8k 僅扣約 1GB，留給模型更多空間。一般對話用 4k，代碼分析用 8k，無特殊需求不要更高。
KV Cache Quantization	Q4_0	對話記憶（KV Cache）本身也佔用顯存，且佔用量與 Context Length 成正比。以 32k context 為例，KV Cache 可能佔用 3–4GB；啟用 Q4_0 量化後縮減至約 1GB，直接騰出 2–3GB 顯存給模型主體。若你使用較短的 context（如 8k），效果會按比例縮小，但仍值得開啟。
Flash Attention	ON	Flash Attention 是一種針對 GPU 記憶體存取模式優化的注意力機制演算法，能減少處理長文本時的顯存峰值，並降低首字延遲（TTFT）。RTX 20 系以上均可使用，在 RTX 30 系（Ampere 架構）上效益最顯著。
Limit Offload to Dedicated VRAM	ON	禁止模型偷用 System RAM 的共用顯存區（Shared GPU Memory）。開啟後，若模型超出 VRAM 上限，LM Studio 會直接報錯，而不是悄悄切換到慢速的 RAM 繼續跑。這確保「能跑就是最高速」，不會出現無聲的效能崩潰。

📊 效能指標解讀
TTFT（Time to First Token）：按下 Enter 到 AI 輸出第一個字的等待時間，主要受 Prompt 處理速度影響。
Tokens per second（t/s）：AI 持續輸出的速度。50+ t/s = 純 VRAM 極速；30+ t/s = 純 VRAM 良好；10–15 t/s = GPU/CPU 混合；5 以下 = 數據正在顯存與 RAM 間搬運。
Shared GPU Memory：應保持為 0。出現數值即代表溢出，須立即降低 Context 或更換量化版本。

第六章：常見問題與除錯手冊

遇到問題時，依照以下分類找到對應的解法，通常能快速恢復正常。

6.1 速度異常

問題：Token/sec 掉到 5 以下（爬行狀態）

原因：顯存溢出（VRAM Overflow）。模型主體加上 KV Cache 超過顯卡容量，Windows 自動啟動了慢速的記憶體交換機制。

解法（請依序嘗試）：

- 先開啟工作管理員，確認「共用 GPU 記憶體（Shared）」是否有數值
- 調低 Context Length（如從 32k 降至 8k）
- 啟用 KV Cache Q4_0 量化，釋放 2–3GB 顯存
- 換用更小的量化版本（如 Q6_K 改為 Q4_K_M）
- 確認「Limit Offload to Dedicated VRAM」已開啟，避免靜默溢出

6.2 程式錯誤

錯誤：Metadata-generation-failed（Python 相關）

原因：Python 版本過新（如 3.13、3.14+），或安裝了 32-bit 版本，導致相依套件無法正確編譯。

解法：前往 python.org 下載 Python 3.12（64-bit）。安裝時勾選「Add Python to PATH」與「Install for all users」，並移除舊版 Python 以避免版本衝突。

錯誤：OpenClaw - 1006 Gateway Closed / 1005 Connection Reset

原因：通常是第三方插件（如 OpenClaw 的 Telegram 插件）後台程序崩潰，多因 Node.js

相依套件未正確安裝。

解法：找到對應的設定檔（如 openclaw.json），將 telegram 或相關插件的啟用選項設為 false，重啟 LM Studio 後再逐一檢查 Node.js 套件是否完整。

6.3 模型輸出異常

現象：回應在 1000 字左右自動截斷

原因：LM Studio 預設的安全輸出長度限制，max_tokens 預設為 1024。

解法：在「Chat」頁面右側的「Model Parameters」中，找到「Response Length / max_tokens」，改為 4096 或 -1（不限制）。

現象：模型突然輸出大量 JSON 或工具調用格式

原因：模型誤判自己處於「工具調用（Function Calling）」情境，尚未切換回自然對話模式。這通常發生在使用預設的 Chat Template 時。

解法：在「System Prompt」欄位中，明確告知模型：「請用自然語言回應，不要使用 JSON 格式或工具調用語法。You are a helpful assistant. Always respond in natural language.」

第七章：操作心法與長期建議

核心哲學：顯存餘裕優先
本地 AI 伺服器的最大原則，就是「讓模型完整住在 VRAM 裡」。建議的顯存分配比例如下：
・**模型主體**：佔 VRAM 的約 70%（例如 12GB 顯卡，模型本體不超過 8–8.5GB）
・**KV Cache**（Context 記憶）：佔 VRAM 的約 20%（使用 Q4_0 量化後通常 1–2GB）
・**系統緩衝與驅動預留**：約 10%（確保不觸發 Shared GPU Memory）
結論：「留有餘裕的顯存」比「強行塞入更大的模型」快 10–20 倍。

7.1 量化選擇原則

- **不要迷信原始精度：**在本地推論環境，Q6_K 或 Q4_K_M 的品質損失幾乎不可感知，卻能大幅降低硬體門檻。
- **Q6_K 是最佳起點：**如果你的顯存裝得下，Q6_K 是兼顧品質與速度的黃金標準。
- **顯存不足時選 Q4_K_M：**速度更快，智商略降 3–5%，但仍遠優於降級到更小的模型。

7.2 大容量 RAM 的正確定位

- **RAM 是你的「實驗室」，不是主力戰場：**96GB RAM 讓你能夠載入一般人跑不動的 70B 超大型模型，但速度只有 1–3 t/s，適合需要高品質推理但不趕時間的任務。
- **日常使用以 9B (Q6_K) 為主力：**完全住在 VRAM 內，30–50 t/s 的速度才是每天用 AI 應有的體驗。

7.3 啟動前快速檢查清單

每次載入新模型或調整設定後，請確認以下項目：

- GPU Offload 設為 -1
- Context Length 是否在合理範圍（日常 4096，代碼 8192）
- KV Cache Quantization 設為 Q4_0
- Flash Attention 已開啟
- Limit Offload to Dedicated VRAM 已勾選
- max_tokens 已設為 4096 或 -1
- 工作管理員確認 Shared GPU Memory = 0

本文件由開發日誌整合修訂而成，適用於 LM Studio + GGUF 本地部署環境。Claude Sonnet 協作。

本文件以 MIT License 全部開源。

Compiled w Love by Ruby Chen, GitHub @RubyChenHaii